

TakaLink

A Mobile-Money Transaction Switch

Object-Oriented Programming — Standalone Lab

Marks: 12 **Language:** Java (JDK 17) **Approach:** Repair an existing codebase, test-driven
You are given a working program and a test file. Submit your .java source files and do not modify the test file.

The job

TakaLink is the country's busiest mobile-money service: tens of millions of wallets, and a nightly *settlement run* that pushes through the day's batch of transfers, cash-outs, merchant payments, and airtime top-ups. A contractor was paid to build the switch that settles that batch; they delivered something that *moves the right amount of money* and passes the company's acceptance tests — and is, by every other measure, a liability. You have been brought in to rebuild it so the company can maintain it, extend it, and *trust it when a transaction fails*. The amounts must stay exactly right. What changes is the *shape* of the code that produces them — and the way it behaves when something goes wrong.

This is a repair job, not a green field

You are handed two things: the contractor's source, and the acceptance test file the company signs off against. The contractor's code **compiles and passes the company's happy-path tests today** — a send between two personal wallets settles to the paisa, a cash-out through an agent likewise. If all you cared about was a batch where nothing ever goes wrong, you could ship it and go home.

But a payment switch is defined by what it does when things *do* go wrong. A wallet has too little balance; a customer fat-fingers their PIN; an account is frozen for fraud review; someone tries an operation their kind of account is not allowed to do. These are not crashes — they are *ordinary, expected outcomes*, and the batch must survive every one of them, settle everything it legitimately can, and hand back a clean account of what settled and what did not. The contractor's engine does none of that. Here is the whole switch as they wrote it — four classes, one of which does everything:

```
class Account {                                // an anemic bag of public fields
    public String type;                        // "PERSONAL", "AGENT", "MERCHANT"
    public double balance;
    public String pin;
    public boolean frozen;
    public double spentToday;
}

class Transaction {                            // another public-field bag
    public String type;                        // "SEND", "CASHOUT", "PAYMENT", "TOPUP"
    public double amount;
    public String fromId, toId, pin;
}

class FeeBook {                                // a second copy of the fee ladder (drifts!)
    static double fee(String txnType, String acctType, double amt) {
        if (txnType == "SEND") return 5.0;
        if (txnType == "CASHOUT") return amt * 0.0185;
        /* ... PAYMENT, TOPUP ... and it disagrees with feeFor() below */
        return 0;
    }
}
```

```

class TransactionEngine {           // the God class
    List<Account> accounts = new ArrayList<>();
    List<Transaction> batch = new ArrayList<>();

    // 0 = OK, 1 = insufficient, 2 = over limit, 3 = bad PIN,
    // 4 = frozen, 5 = operation not allowed for this account
    int process(Transaction t) {
        Account from = find(t.fromId), to = find(t.toId);
        // ---- first: which transaction? then: which account? ----
        if (t.type == "SEND") {
            if (from.frozen) return 4;
            if (!from.pin.equals(t.pin)) return 3;
            from.balance -= t.amount;           // money leaves NOW...
            if (from.spentToday + t.amount > limit(from)) return 2; // ...
            too late
            if (from.balance - 5.0 < 0) return 1; // ...already overdrawn
            from.balance -= 5.0;
            to.balance += t.amount;
            from.spentToday += t.amount;
        } else if (t.type == "CASHOUT") {
            if (from.type == "MERCHANT") return 5; // merchants can't cash
            out
            double fee = feeFor(t);               // FeeBook says 0.0185;
            this says 0.019
            /* ... yet another account-type ladder, more half-updates ... */
        } else if (t.type == "PAYMENT") { /* ... */ }
        else if (t.type == "TOPUP") { /* ... */ }
        return 0;
    }
    double feeFor(Transaction t) { if (t.type == "SEND") ... else if ... }
    double limit(Account a)       { if (a.type == "PERSONAL") ... else if ... }
    }
    void settleBatch() {
        for (Transaction t : batch) {
            int code = process(t);
            if (code != 0) System.out.println("failed: " + code);
        }
    }
}

```

It worked on delivery day. Then the company tried to *run a real night's batch* through it, and every method above became a place where money and truth go missing:

The contractor's version, and why it hurts

- **Every settlement asks two questions first.** Before process can move a paisa it must ask “which kind of transaction is this?” and then, inside almost every branch, “which kind of account is this?” The same two-level if ladder is copied into process, feeFor, and limit — and they drift: feeFor charges a cash-out at 0.019 while FeeBook charges 0.0185, and nobody can say which one is “the” fee.
- **Adding anything means editing everything.** A new “government disbursement” account, or a new “bill payment” transaction, has no single home. You must open process, feeFor, limit, and FeeBook and thread one more else if through each, in the right order, without a mistake.
- **Failure is an int you can forget to check.** process reports trouble by returning a number,

and `settleBatch` only *prints* it. A caller who forgets to read the code sails on as though the money moved. And the number tells you *that* it failed, never gives you a place to attach *why* in a form the next layer can act on — “2” means over-limit today and something else next release.

- **A failed transaction still moves money.** Look at the `SEND` branch: `from.balance -= t.amount` runs *before* the limit check and *before* the balance check. A transaction that is about to be rejected has *already* debited the sender; the credit to the receiver may or may not follow. The batch can end with money created or destroyed — the one thing a payment switch may never do.
- **A half-built object is a legal object.** `new Account()` with a null type, a null pin, and a balance of -500 compiles and settles. `new Transaction()` with `amount = -1000` is a “send” that *pulls* a thousand taka out of the recipient — a theft the type system waves through.
- **The fields are wide open.** `balance`, `pin`, `frozen`, and `spentToday` are all public. Any code holding an `Account` can write `a.balance = 1_000_000`, read the PIN in the clear, or set `a.frozen = false` on a flagged account. The PIN check and the freeze protect nothing.

Your task is to rebuild TakaLink so that none of the six pains above can happen again, while every settled amount comes out to the same number. **With one deliberate exception noted below, nothing in this document tells you which Java features to use.** It tells you how the payment world behaves; the shape of your classes is the thing you are being marked on. But read the six pains as a promise: if your rebuild keeps the contractor’s shape, it will hit the same six walls — and the test file will catch it at the first one.

The one instruction we *do* give you: fail with exceptions

This lab is about the moment a transaction cannot go through. Everywhere else you are free to choose your own design; *here* we are explicit, because signalling failure well is the skill being taught:

▷ Notice

A transaction that cannot legitimately settle must **throw a checked exception**, not return a status code, not print, and not silently half-settle. The settlement run must **catch** those exceptions, one transaction at a time, record what went wrong, and *carry on with the rest of the batch*. Use Java’s exception mechanism — your own exception *types*, throws on the methods that can fail, and `try/catch` in the one place that handles them.

Two different kinds of “wrong” show up in this system, and they call for two different tools — keep them apart:

- **A broken request — data that should never have been built at all.** A negative amount, a null wallet id, a null PIN, an opening balance below zero. This is a programming or data error, caught the instant the object is constructed. Reject it from the *constructor* with an `IllegalArgumentException` (unchecked) — a malformed `Transaction` or `Account` must not come into existence in the first place.
- **A well-formed request that the business *refuses* — a failure that is expected and must be handled.** The wallet is short of funds; the PIN is wrong; the account is frozen; the daily limit is spent; the operation is not one this kind of account may perform. These are the normal weather of a payment system. Model them as a small family of **checked** exceptions under one common parent, thrown at settlement time and caught by the run.

Why *checked*? Because “this transfer might be refused” is not an afterthought a caller may forget — it is part of the contract of settling a transaction, and the compiler should force every caller to decide what to do about it. A checked exception on the “settle” operation makes the failure *impossible to ignore*, which is exactly the property the contractor’s int code failed to give you. When you are unsure whether the beginner’s-guide handout applies, read **A Beginner’s Guide to Exception Handling in Java**, shipped with this lab, before you start.

How the payment world behaves

Read these as facts about the business. Each one quietly decides something about how your classes must relate. The *italic* and **bold** phrases are load-bearing — they are the hints about which things are the same underneath and which merely differ on the surface.

The wallet and its owner

- Every party on the switch holds a **wallet**: an identity, a balance in taka, a secret PIN, a frozen/active status, and a running total of *how much it has already sent today*. The balance is money the wallet *holds*; nobody outside the wallet may set it directly — it changes *only* as the result of a transaction the wallet itself carries out.
- A wallet is asked to *take money out* (debit) and to *take money in* (credit). A debit that would drop the balance below zero is refused; a credit of a non-positive amount is nonsense and is refused. *The wallet is the only thing that may touch its own balance*, and it applies its own rules every time it is asked.
- A wallet checks its own PIN and its own frozen status. *Whether a caller supplied the right PIN, and whether the wallet is frozen, are questions only the wallet can answer* — the PIN is never handed out to be compared elsewhere.
- “How much may this wallet still send today” is *worked out on demand* from its daily limit and what it has already sent; it is never a separate figure someone keeps in step by hand.

The account categories — where the wallet’s rules split

There is no such thing as a “plain” account. *Every wallet is of a definite category*, and the category is not a label you read off a field — it **is what the account fundamentally is**, and it decides three things that genuinely differ from one kind of account to the next:

- a **personal** account is an ordinary customer wallet: a modest daily sending limit, and it may do any of the four transactions as the paying side;
- an **agent** account is a cash-out point: a much larger daily limit, and it is the *counterparty* that hands over cash when a customer cashes out;
- a **merchant** account exists to *receive* payments: it may not send money and may not cash out at all, and any attempt to make it do so is refused.

Everything else about a wallet — holding a balance, debiting and crediting under the zero-floor rule, checking its PIN, checking its frozen status, tracking today’s spend — is done **identically for every category**. *These three are not three unrelated classes and they are not one class with a label* — they are three **particular kinds of the same underlying wallet**, alike in everything just listed and different only in their limit and in which operations they are permitted to perform.

▷ Notice

When the regulator adds a fourth category next year — say a “government disbursement” wallet — there must be exactly **one** place to describe it: its limit and its permitted operations. Nothing that debits, credits, checks a PIN, settles a batch, or totals the run should have to be reopened. If adding a category makes you edit the settlement loop, your rebuild has the contractor’s shape.

The transaction types — where the *behaviour* splits

Every transaction shares the same core: a paying wallet, a receiving wallet, an amount, and the PIN offered by the payer. Beyond that core, the four kinds differ in two behaviours — *what they cost* and *how they move the money*:

- a **send** moves money from one wallet to another and costs a *flat fee* charged to the sender;
- a **cash-out** pulls money from a customer through an agent and costs a *percentage* of the amount (so a larger cash-out genuinely costs more);
- a **payment** pays a merchant and is *free to the payer*;
- a **top-up** buys airtime and is *free* as well.

How a transaction is priced, and exactly which wallets it debits and credits, is the one thing that differs from kind to kind — but the **skeleton of settling is always the same**: confirm the payer’s PIN, confirm the payer is not frozen, confirm the payer is allowed this operation, confirm the amount and fee are within the payer’s remaining daily limit and balance, and only *then* move the money and record the spend. *That skeleton is written down once*; each kind of transaction fills in only its own fee and its own movement of money.

Notice the two axes. “Which *kind of transaction*” and “which *kind of account*” are the two questions the contractor asked, nested, in every method. They are *two different variations*, and they belong in *two different families of classes* that meet only at settlement time — a transaction asks a wallet to debit, credit, verify its PIN, and confirm the operation is allowed, and the wallet answers in its own category’s voice. Neither family should contain a ladder that enumerates the other.

Settlement must be all-or-nothing

- A single transaction either settles **completely** — payer debited by amount *and* fee, receiver credited, the payer’s daily spend advanced — or it **does not touch a single balance**. There is no in-between. *Every reason a transaction might be refused must be checked **before** any wallet is debited or credited*, so that a refusal throws its exception with every balance exactly as it was.
- This is the direct cure for the contractor’s “money leaves now, we check later.” Validate first; move money last; and if validation throws, nothing has moved.

The settlement run (the nightly “batch”)

- The run holds a batch of transactions — sends, cash-outs, payments, top-ups, all together in *one* place. From the run’s point of view they are simply “transactions to be settled,” and it must **hold any of them side by side** without caring which kind each one is.
- **Settle the whole batch in one sweep**. Executing the run attempts *every* transaction it holds in a single pass — “for each transaction waiting, settle it” — where each transaction **already knows how to settle itself**. That loop must not contain a ladder asking each transaction which kind it is.

- **Survive every failure.** When a transaction throws, the run *catches it*, records the transaction and the reason it was refused, and **moves on to the next one**. One insufficient-balance transfer must never abort the settlement of the thousand good transactions behind it.
- **Hand back an honest report.** After the sweep the run reports what *settled* and what was *rejected* — how many of each, the total value moved, the total fees collected, and, for every rejected transaction, the reason it failed. That report is the switch’s account of the night.
- **One name, two ways to submit.** Adding a transaction to the run should be possible either on its own, or together with a repeat count that queues the same transaction several times (a standing instruction settled once per day for several days). Same idea, same name, different inputs — the caller should not have to remember two method names for “add this to tonight’s batch.”
- The run can also say how many transactions are currently waiting.

Decide the relationships before you type. Every bold and italic phrase above — “holds a wallet,” “the only thing that may touch its own balance,” “worked out on demand,” “particular kinds of the same underlying wallet,” “two different families of classes that meet only at settlement,” “already knows how to settle itself,” “catches it . . . and moves on” — is describing a relationship between two of your classes, or the flow of one exception. Work them out on paper *before* you open the editor.

The money arithmetic, worked once so you are sure of it

There is no slab arithmetic here, but the fee-and-atomicity rule trips people up, so here it is spelled out. A settlement debits the payer for *amount plus fee*, credits the receiver for the *amount*, and the fee is the switch’s revenue.

A send of 1000 taka (flat fee 5.0). The sender is debited $1000 + 5.0 = 1005.0$; the receiver is credited 1000.0; revenue rises by 5.0; the sender’s spend-today rises by 1000. For this to settle, the sender must (a) not be frozen, (b) offer the right PIN, (c) have `spentToday + 1000` within the sender’s daily limit, and (d) hold at least 1005.0. If *any* of those fails, the transaction throws and *no* balance changes.

A cash-out of 2000 taka (fee 1.85%). The fee is $2000 \times 0.0185 = 37.0$. The customer is debited $2000 + 37.0 = 2037.0$; the agent is credited 2000.0 (the cash they hand over); revenue rises by 37.0. If the customer holds only 2000, the debit of 2037 would breach the zero floor — the transaction throws *insufficient balance*, and crucially the customer’s balance is *still* 2000 afterwards, not -37 , because nothing moved.

▷ Notice

The contractor’s bug was ordering: subtract first, discover the problem second. Your rule is the reverse — *compute the fee, confirm every precondition, and only then debit and credit*. Write the “move the money” step so that it is physically unreachable unless every check has already passed. The exact fee, limit, and starting-balance figures are fixed in the test file; the *shape* of the sum and the *order* of the checks are what matter here.

Step 0 — Sketch the interactions (do this first, and submit it)

Before any code, draw — by hand is fine — how these objects hand work to one another when the nightly run settles one transaction. Show, for example: who the settlement run talks to when you

execute it; what a cash-out has to ask of the customer’s wallet and of the agent’s wallet before any money moves; where the PIN check and the frozen check happen and *who* answers them; and *what travels back up* to the run when a transaction is refused instead of settling. Use boxes and arrows and your own labels; **this is not UML and needs no formal notation**. Photograph or scan it and submit it with your code. You will be asked to talk through it.

Question Your Viva Will Ask (the sketch)

Point at the single arrow in your drawing that carries a *refusal* back to the run — the thing that travels up when a wallet is too short to pay. Is it the same kind of arrow as a normal return value, or a different mechanism entirely? And point at the box where that refusal *stops* travelling: who decides that one failed transaction does not become the whole batch’s failure? If your drawing carries a refusal back as a number that the caller might not read, you have drawn the contractor’s version again.

What each class must expose

Here is the catalogue of behaviours your classes must offer, by *name and effect only*. Notice what is deliberately missing: we are **not** telling you any method’s parameters or its return type, and we are not telling you which class sits above which. The provided test file is your specification for the exact signatures; the relationships are yours to design from the rules above.

- **A wallet**, whatever its category, can: report its balance; *debit* and *credit* itself under the zero-floor and positive-amount rules; verify a PIN offered to it; report whether it is frozen; and report how much it may still send today. It refuses, at construction, a negative opening balance, a null or blank id, or a null PIN. A wallet with no category cannot be constructed.
- **A personal, an agent, and a merchant wallet** each answer “what is my daily sending limit?” and “am I allowed to be the payer of *this* kind of transaction?” *in their own way*; everything else they share *without restating it*.
- **Every transaction**, whatever its kind, can: report its fee; and *settle itself* against the wallets it names — running the common check-then-move skeleton, throwing the appropriate checked exception if any precondition fails, and moving no money if it throws. It refuses, at construction, a non-positive amount or a null wallet reference.
- **A send, a cash-out, a payment, and a top-up** each provide their own fee and their own movement of money; the settlement skeleton around them is written once and shared.
- **The checked exception family**: one common parent for “a transaction was refused,” with distinct kinds for at least — insufficient balance, daily-limit exceeded, wrong PIN, frozen account, and operation-not-allowed-for-this-account. Each carries enough information to explain itself in the run’s report.
- **The settlement run** can: take a transaction to be settled (and, by the same name, take a transaction together with a repeat count); *execute* the whole batch, catching each refusal and continuing; report how many settled and how many were rejected, the total value moved and total fees collected, and the reason each rejected transaction failed; and report how many transactions are waiting.

▷ Notice

The test file constructs each kind of wallet and each kind of transaction *by its own name*, drops a mixture into one settlement run, and asserts on both the *amounts* and the *failures*. A design that funnels every transaction through one engine with a “which kind am I” field, or that signals failure with anything other than a thrown exception, will not pass — and that, precisely, is the

lesson. Read the test file the way you have been taught: let it tell you each method's return type, parameters, and *which exception each failure throws*; let *this* document tell you how the classes relate.

What the test file checks (in words — read the file for the types)

You will be given `TakaLinkTest.java`. You do not modify it. The following is what it verifies; the exact signatures, fees, limits, and starting balances are in the file.

- **The wallet guards itself.** A negative opening balance, a null id, or a null PIN is refused at construction. A debit that would go below zero is refused; a credit of a non-positive amount is refused; a correct debit/credit pair leaves the arithmetic exact.
- **Each transaction charges its own fee and moves money its own way.** A send charges the flat fee to the sender; a cash-out charges the percentage; a payment and a top-up are free. Each moves money between exactly the wallets it should.
- **Settlement is all-or-nothing.** A send that fails on insufficient balance, on the daily limit, on a wrong PIN, or on a frozen account leaves *both* wallets' balances exactly as they were before the attempt — checked by reading the balances after the throw.
- **Each failure throws its own checked exception.** Insufficient balance, daily-limit exceeded, wrong PIN, frozen account, and an operation the account is not allowed to perform each throw their *distinct* exception type, all catchable as the one common parent.
- **The account categories differ only where they should.** A merchant refused as the sender of a send or a cash-out throws operation-not-allowed; a personal and an agent wallet accept up to their *different* daily limits; everything else behaves identically across categories.
- **The run settles a mixed batch and survives failures.** A batch containing good transactions and several that will be refused settles every good one, rejects every bad one with the right reason, and the totals (count settled, count rejected, value moved, fees collected) add up by hand.
- **One slot, many forms.** A single variable is pointed at a send, then a cash-out, then a payment; each time it is submitted and later settled correctly — the same variable standing in for any kind of transaction.
- **Submit once, or submit repeated.** Submitting a transaction with a repeat count queues it that many times; submitting it plainly queues it once — same name, two input shapes.

Question Your Viva Will Ask (the dispatch)

Your settlement loop asks every transaction the *same* question under the *same* name — “settle yourself” — yet a send moves money one way and a cash-out another, and each may throw a *different* exception. The loop was written before tonight's batch existed. *Who* picks the right money-movement and the right fee for each transaction, and *when* — when you wrote the loop, or while it runs? Now contrast the two ways of submitting a transaction (plain, or with a repeat count): which of those two is chosen by the same run-time mechanism, and which is settled earlier, by the compiler, from the way the call is written?

Marking Rubric

#	Criterion	Marks
1	The four transaction kinds each price and move money in their own way, held together in one settlement run through a single common type, settled by one branch-free sweep	2.5
2	The three account categories each define their limit and permitted operations in their own way, sharing all common wallet behaviour without restating it; a category-less wallet cannot be constructed	2.5
3	Failures are signalled through a checked exception family (one parent, distinct kinds), <i>thrown</i> not returned; the run catches each and continues, producing a settled/rejected report with reasons	2.5
4	Settlement is atomic: every precondition is checked before any balance moves, so a thrown exception leaves all balances untouched	1.5
5	Encapsulation: balance, PIN, and frozen status are sealed against outside mutation and the PIN is never handed out; a malformed transaction or wallet is rejected at construction with <code>IllegalArgumentException</code>	1.5
6	Submitting offers one name with two input shapes (single, and repeated)	0.5
7	Wallet modelled as something an account category <i>is</i> a kind of, and a transaction as something that <i>has</i> the wallets it names; clear informal interaction sketch submitted	1.0
Total		12

▷ Notice

Passing the tests is necessary but not sufficient. Some properties cannot be written as an assertion — that a category-less wallet is impossible to construct, that the settlement skeleton is written in exactly one place, and that failure travels only as a thrown exception and never as a returned code — and the is-a / has-a choices are a matter of design, not output. Criteria 2, 3, 5, and 7 are judged by reading your code and by the viva. A solution that reaches the right numbers through one engine and a forest of `if/else` on a type field, or that signals refusal with an `int`, has rebuilt the contractor's version and will score accordingly, however green the test bar looks.

Ground rules

- Java 17. Use `double` for money and `int` for counts.
- Use a `java.util.ArrayList` for the batch and for any other growing collection (the account registry, the settled and rejected lists in your report). You have been taught `ArrayList`; use it here rather than a fixed-size array with a running count. The container is not the point of this lab; *what* you can put in it, and *how it behaves when a transaction fails*, is. Whatever you store it in, the run must hold every kind of transaction side by side under one common type and settle them without ever asking which kind each one is.

- The amounts must not change. Every fee, limit, and starting balance the contractor used is fixed in the test file; your rebuild must reproduce them exactly. You are changing the *shape* of the code and the *way it fails*, not the money.
- Say every relationship aloud before you commit to it: “*is a*” versus “*has a*.” A personal wallet *is a* wallet; a transaction *has* the wallets it names. And say every failure aloud: is it a broken *request* (reject at construction, unchecked) or a refused *transaction* (throw a checked exception, caught by the run)?
- Partial credit is awarded; submit whatever compiles. Submit your source files and your Step-0 sketch.